



Complete Docker Compose Setup Documentation



Pet Management Application - Production Deployment Guide



Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 3. [Prerequisites](#)
 4. [Project Structure](#)
 5. [Docker Compose Configurations](#)
 6. [Deployment Guide](#)
 7. [Service Management](#)
 8. [Monitoring & Maintenance](#)
 9. [Troubleshooting](#)
 10. [Security & Performance](#)
 11. [Production Checklist](#)
-



Overview

This documentation provides a complete guide for deploying the Pet Management Application using Docker Compose. The application features a modern three-tier architecture with Angular frontend, Node.js backend, and MongoDB database, all containerized for easy deployment and scalability.

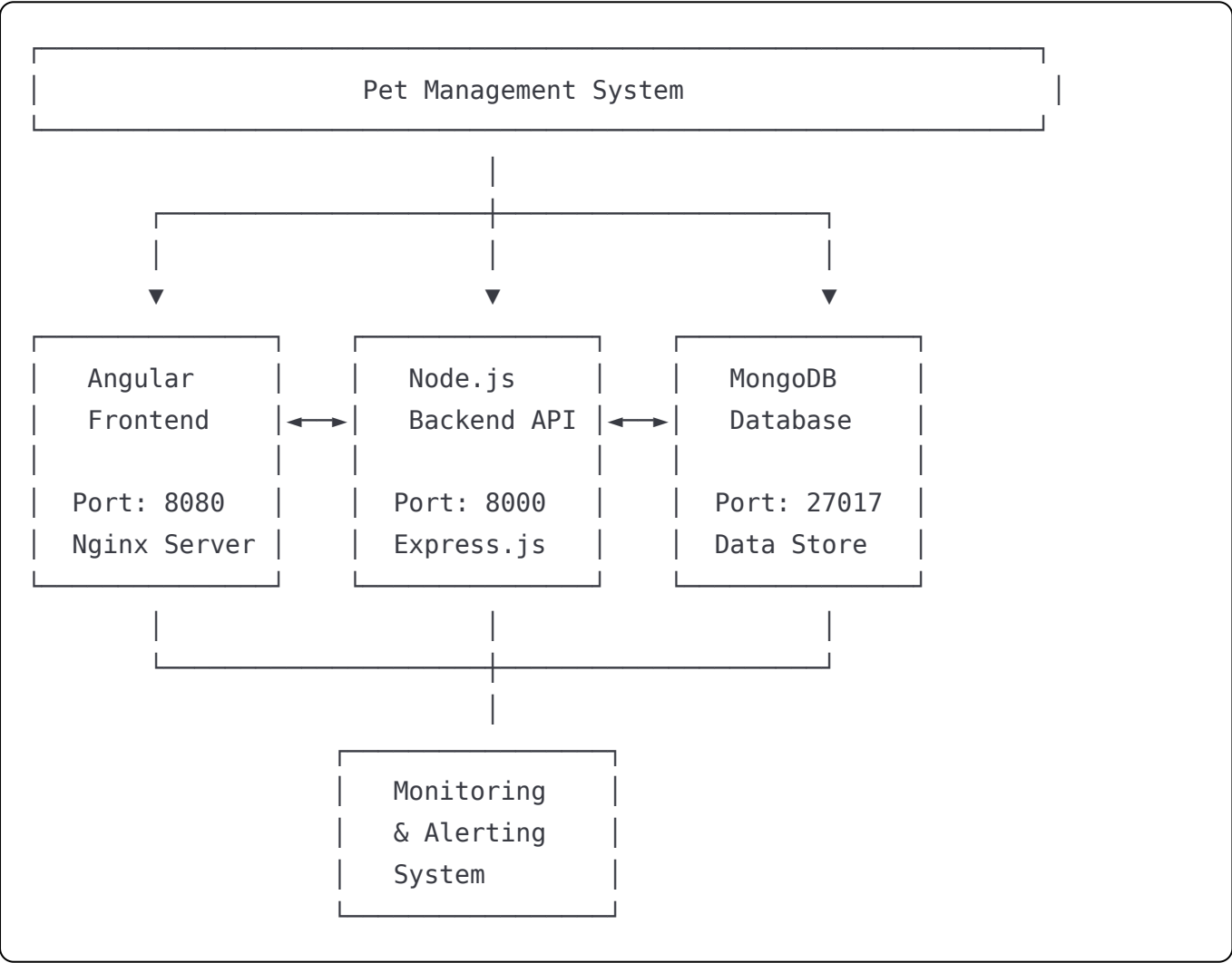
Key Features

-  **Microservices Architecture:** Separate deployments for frontend and backend
-  **Production-Ready Containers:** Optimized builds with multi-stage Docker images
-  **Database Integration:** MongoDB 7 with persistent storage

-  **Enterprise Monitoring:** Automated health checks and alerting system
-  **Security Hardening:** JWT authentication, bcrypt encryption, CORS policies
-  **Scalable Design:** Container orchestration ready for production
-  **Comprehensive Documentation:** Complete deployment and maintenance guides

Architecture

System Architecture Diagram



Component Details

Component	Technology	Port	Purpose
Frontend	Angular 18 + Nginx	8080	User interface and client-side logic
Backend	Node.js + Express	8000	REST API and business logic
Database	MongoDB 7	27017	Data persistence and storage

Component	Technology	Port	Purpose
Monitoring	Bash Script	N/A	Health checks and alerting

Network Architecture

- **Backend Network:** Isolated network for backend and database communication
- **Frontend Network:** Separate network for frontend service
- **Bridge Driver:** Default Docker bridge networking for inter-container communication

✓ Prerequisites

System Requirements

Operating System:

- Linux (Ubuntu 20.04+, CentOS 8+, Debian 11+)
- macOS 11.0+
- Windows 10+ with WSL2

Software Requirements:

```
bash

# Docker
Docker Engine: 20.10.0 or higher
Docker Compose: 1.29.0 or higher (or Docker Compose V2)

# System Resources
CPU: 2+ cores recommended
RAM: 4GB minimum, 8GB recommended
Disk: 10GB free space minimum
```

Installation Verification

```
bash
```

```
# Check Docker version
docker --version
# Expected: Docker version 20.10.x or higher

# Check Docker Compose version
docker-compose --version
# Expected: docker-compose version 1.29.x or higher

# Verify Docker is running
docker ps
# Should show running containers or empty list

# Check available resources
docker system info | grep -E "CPUs|Total Memory"
```

Required Permissions

```
bash

# Add user to docker group (Linux)
sudo usermod -aG docker $USER
newgrp docker

# Verify permissions
docker run hello-world
```

Project Structure

Complete Directory Layout

```
petmanagement_project/
├── petmanagement_backend_rajalakshmi/
│   ├── docker-compose.yml      # Backend + MongoDB orchestration
│   └── Dockerfile              # Backend container build
├── instructions
│   ├── .env                   # Environment variables (gitignored)
│   ├── .dockerignore          # Docker build exclusions
│   ├── server.js              # Express application entry point
│   ├── package.json           # Node.js dependencies
│   ├── package-lock.json      # Locked dependency versions
│   └── app/
│       ├── config/
│       └── db.config.js        # Database connection settings
```

```

├── auth.config.js           # JWT configuration
├── models/
│   ├── index.js           # Mongoose connection setup
│   ├── user.model.js      # User schema
│   └── pet.model.js        # Pet report schema
├── controllers/
│   ├── auth.controller.js  # Authentication logic
│   └── pet.controller.js   # Pet management logic
├── routes/
│   ├── auth.routes.js     # Auth endpoints
│   └── pet.routes.js      # Pet endpoints
├── middleware/
│   ├── authJwt.js         # JWT verification
│   └── verifySignUp.js    # Registration validation
├── petmanagement_frontend_rajalakshmi/
│   ├── docker-compose.yml  # Frontend orchestration
│   ├── Dockerfile          # Multi-stage Angular build
│   ├── nginx.conf          # Nginx web server config
│   ├── .dockerignore       # Build exclusions
│   ├── angular.json        # Angular CLI configuration
│   ├── package.json        # Frontend dependencies
│   ├── tsconfig.json       # TypeScript configuration
│   └── src/
│       ├── app/
│       │   ├── components/ # Angular components
│       │   ├── services/   # API services
│       │   ├── guards/     # Route guards
│       │   └── models/     # TypeScript models
│       └── environments/
│           ├── environment.ts # Development config
│           └── environment.prod.ts # Production config
├── docker-compose.yml       # Full stack orchestration (optional)
├── monitor-petfinder.sh    # Monitoring and alerting script
├── README.md               # Project documentation
└── .gitignore              # Version control exclusions

```

Key Configuration Files

Backend .env File Template

```
env

# Database Configuration
DB_HOST=mongodb
DB_PORT=27017
DB_NAME=petmanagement

# Application Configuration
NODE_ENV=production
PORT=8000
API_BASE_URL=http://localhost:8000

# JWT Configuration
JWT_SECRET=your-super-secret-jwt-key-change-in-production
JWT_EXPIRATION=86400

# CORS Configuration
CORS_ORIGIN=http://localhost:8080
```

Frontend Environment Configuration

```
typescript

// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'http://localhost:8000/v1'
};
```



Docker Compose Configurations

1. Backend Docker Compose

File: `petmanagement_backend_rajalakshmi/docker-compose.yml`

```
yaml
```

version: '3.8'

services:

MongoDB Database Service

mongodb:

image: mongo:7-jammy

container_name: petmanagement-backend-mongodb

restart: unless-stopped

ports:

- "27017:27017"

environment:

MONGO_INITDB_DATABASE: petmanagement

volumes:

- mongodb_data:/data/db

- mongodb_config:/data/configdb

networks:

- backend-network

healthcheck:

test: echo 'db.runCommand("ping").ok' | mongosh localhost:27017/test --qu

interval: 30s

timeout: 10s

retries: 3

start_period: 40s

Backend API Service

backend:

build:

context: .

dockerfile: Dockerfile

container_name: petmanagement-backend

restart: unless-stopped

ports:

- "8000:8000"

environment:

- NODE_ENV=production

env_file:

- .env

depends_on:

mongodb:

condition: service_healthy

networks:

- backend-network

volumes:

- ./app

- /app/node_modules

healthcheck:

```
test: ["CMD", "curl", "-f", "http://localhost:8000/"]
interval: 30s
timeout: 10s
retries: 3
start_period: 40s

volumes:
  mongodb_data:
    driver: local
  mongodb_config:
    driver: local

networks:
  backend-network:
    driver: bridge
```

Configuration Breakdown:

Service	Configuration	Purpose
mongodb	mongo:7-jammy	Latest stable MongoDB on Ubuntu Jammy
	restart: unless-stopped	Auto-restart on failure
	volumes: mongodb_data	Persistent database storage
	healthcheck	Automated health monitoring
backend	build: context: .	Build from local Dockerfile
	depends_on: mongodb	Wait for database before starting
	env_file: .env	Load environment variables
	volumes: /app/node_modules	Optimize node_modules mounting

2. Frontend Docker Compose

File: petmanagement_frontend_rajalakshmi/docker-compose.yml

```
yaml
```



```
version: '3.8'

services:
  # Frontend Application Service
  frontend:
    build:
      context: .
      dockerfile: Dockerfile
      args:
        - NODE_ENV=production
    container_name: petmanagement-frontend
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      - NODE_ENV=production
    networks:
      - frontend-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:80/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s

networks:
  frontend-network:
    driver: bridge
```

Configuration Breakdown:

Property	Value	Purpose
Image	Multi-stage build	Optimized production image
Port Mapping	8080:80	Host port 8080 to container port 80
Restart Policy	unless-stopped	Automatic recovery
Healthcheck	HTTP GET request	Service availability monitoring

3. Full Stack Docker Compose (Optional)

File: docker-compose.yml (project root)

yaml

```
version: '3.8'
```

```
services:
```

```
  # MongoDB Database
```

```
  mongodb:
```

```
    image: mongo:7-jammy
```

```
    container_name: petmanagement-mongodb
```

```
    restart: unless-stopped
```

```
    ports:
```

```
      - "27017:27017"
```

```
    environment:
```

```
      MONGO_INITDB_DATABASE: petmanagement
```

```
    volumes:
```

```
      - mongodb_data:/data/db
```

```
      - mongodb_config:/data/configdb
```

```
    networks:
```

```
      - petmanagement-network
```

```
  healthcheck:
```

```
    test: echo 'db.runCommand("ping").ok' | mongosh localhost:27017/test --quiet
```

```
    interval: 30s
```

```
    timeout: 10s
```

```
    retries: 3
```

```
  # Backend API
```

```
  backend:
```

```
    build:
```

```
      context: ./petmanagement_backend_rajalakshmi
```

```
      dockerfile: Dockerfile
```

```
    container_name: petmanagement-backend
```

```
    restart: unless-stopped
```

```
    ports:
```

```
      - "8000:8000"
```

```
    environment:
```

```
      - NODE_ENV=production
```

```
    env_file:
```

```
      - ./petmanagement_backend_rajalakshmi/.env
```

```
    depends_on:
```

```
      mongodb:
```

```
        condition: service_healthy
```

```
    networks:
```

```
      - petmanagement-network
```

```
    volumes:
```

```
      - ./petmanagement_backend_rajalakshmi:/app
```

```
      - /app/node_modules
```

```
  healthcheck:
```

```
    test: ["CMD", "curl", "-f", "http://localhost:8000/"]
```

```
    interval: 30s
    timeout: 10s
    retries: 3

# Frontend Application
frontend:
  build:
    context: ./petmanagement_frontend_rajalakshmi
    dockerfile: Dockerfile
  container_name: petmanagement-frontend
  restart: unless-stopped
  ports:
    - "8080:80"
  environment:
    - NODE_ENV=production
  depends_on:
    backend:
      condition: service_healthy
  networks:
    - petmanagement-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:80/"]
    interval: 30s
    timeout: 10s
    retries: 3

volumes:
  mongodb_data:
    driver: local
  mongodb_config:
    driver: local

networks:
  petmanagement-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16
```

Deployment Guide

Phase 1: Environment Preparation

Step 1.1: System Requirements Check

```
bash

# Verify Docker installation
docker --version
# Expected: Docker version 20.10.x or higher

docker-compose --version
# Expected: docker-compose version 1.29.x or higher

# Check system resources
docker system info | grep -E "CPUs|Total Memory|Operating System"

# Verify Docker daemon is running
systemctl status docker # Linux
# or
docker info             # All platforms
```

Step 1.2: Clean Existing Containers (if needed)

```
bash

# Stop all running containers
docker stop $(docker ps -q) 2>/dev/null || true

# Remove all containers
docker rm $(docker ps -aq) 2>/dev/null || true

# Remove unused networks
docker network prune -f

# Remove unused volumes (CAUTION: This removes data)
docker volume prune -f

# Clean up unused images
docker image prune -a -f
```

Step 1.3: Create Monitoring Infrastructure

```
bash
```

```
# Create log directories
sudo mkdir -p /var/log/petmanagement
sudo touch /var/log/petfinder.log
sudo touch /var/log/petfinder-alerts.log

# Set appropriate permissions
sudo chmod 666 /var/log/petfinder.log
sudo chmod 666 /var/log/petfinder-alerts.log

# Make monitoring script executable
chmod +x monitor-petfinder.sh

# Test script functionality
./monitor-petfinder.sh --check
```

Phase 2: Backend Deployment

Step 2.1: Navigate to Backend Directory

```
bash

cd petmanagement_backend_rajalakshmi
```

Step 2.2: Configure Environment Variables

```
bash
```

```
# Create .env file from template
cat > .env << 'EOF'
# Database Configuration
DB_HOST=mongodb
DB_PORT=27017
DB_NAME=petmanagement

# Application Configuration
NODE_ENV=production
PORT=8000

# JWT Configuration
JWT_SECRET=$(openssl rand -base64 32)
JWT_EXPIRATION=86400

# CORS Configuration
CORS_ORIGIN=http://localhost:8080
EOF

# Verify .env file
cat .env
```

Step 2.3: Validate Configuration

```
bash

# Validate Docker Compose syntax
docker-compose config

# Check for configuration errors
docker-compose config --quiet && echo "Configuration valid" || echo "Configuration invalid"

# Verify Dockerfile exists
ls -la Dockerfile

# Check database configuration
cat app/config/db.config.js
```

Step 2.4: Build and Deploy Backend Services

```
bash
```

```
# Option 1: Build and run with logs visible
docker-compose up --build

# Option 2: Build and run in detached mode (recommended)
docker-compose up -d --build

# Monitor build progress
docker-compose logs -f
```

Step 2.5: Verify Backend Deployment

```
bash

# Check service status
docker-compose ps

# Expected output:
#           Name                                Command                                State          P
# -----
# petmanagement-backend                        docker-entrypoint.sh npm start        Up             0.0.0.0
# petmanagement-backend-mongodb                docker-entrypoint.sh mongod           Up             0.0.0.0

# Check container health
docker ps --filter "name=petmanagement-backend" --format "table {{.Names}}\t{{.
# View backend logs
docker-compose logs backend --tail=50

# View MongoDB logs
docker-compose logs mongodb --tail=50
```

Step 2.6: Test Backend API Endpoints

```
bash
```

```
# Test 1: Health Check
curl http://localhost:8000/
# Expected: {"status":true,"message":"Welcome to pet missing report management"}

# Test 2: User Registration
curl -X POST http://localhost:8000/v1/users/signup \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Test User",
    "email": "test@example.com",
    "password": "Password123!"
  }'
# Expected: Success response with user data

# Test 3: User Login
curl -X POST http://localhost:8000/v1/users/signin \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test@example.com",
    "password": "Password123!"
  }'
# Expected: JWT token response

# Test 4: Database Connection
docker exec petmanagement-backend-mongodb mongosh --eval "db.adminCommand('ping')"
# Expected: { ok: 1 }

# Test 5: Get All Pet Reports (requires authentication)
TOKEN=$(curl -s -X POST http://localhost:8000/v1/users/signin \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"Password123!"}' \
  | jq -r '.token')

curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/v1/users/get_all_pet_reports
```

Phase 3: Frontend Deployment

Step 3.1: Navigate to Frontend Directory

```
bash
cd ../petmanagement_frontend_rajalakshmi
```

Step 3.2: Validate Configuration


```
bash

# Validate Docker Compose syntax
docker-compose config

# Verify required files exist
ls -la Dockerfile nginx.conf angular.json

# Check production environment configuration
cat src/environments/environment.prod.ts
```

Step 3.3: Build and Deploy Frontend Service

```
bash

# Option 1: Build and run with logs visible
docker-compose up --build

# Option 2: Build and run in detached mode (recommended)
docker-compose up -d --build

# Monitor build progress (multi-stage build takes time)
docker-compose logs -f frontend
```

Step 3.4: Verify Frontend Deployment

```
bash

# Check service status
docker-compose ps

# Expected output:
#           Name                                Command                                State          Ports
# -----
# petmanagement-frontend  /docker-entrypoint...  Up            0.0.0.0:8080->80/tcp

# Check container health
docker ps --filter "name=petmanagement-frontend" --format "table {{.Names}}\t{{.Health}}\n"

# View frontend logs
docker-compose logs frontend --tail=50
```

Step 3.5: Test Frontend Application

```
bash
```

```
# Test 1: Check if Nginx is serving content
curl -s http://localhost:8080/ | head -20
# Expected: <!DOCTYPE html> with Angular application

# Test 2: Verify static assets
curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/
# Expected: 200

# Test 3: Check Nginx configuration
docker exec petmanagement-frontend nginx -t
# Expected: nginx: configuration file /etc/nginx/nginx.conf test is successful

# Test 4: Verify Angular build artifacts
docker exec petmanagement-frontend ls -la /usr/share/nginx/html/
# Expected: index.html, main.js, polyfills.js, etc.

# Test 5: Open in browser
echo "Open http://localhost:8080 in your browser"
```

Phase 4: Integration Testing

Step 4.1: Test Frontend-Backend Communication

```
bash

# Test API call from frontend perspective
curl -X POST http://localhost:8000/v1/users/signup \
  -H "Content-Type: application/json" \
  -H "Origin: http://localhost:8080" \
  -d '{
    "name": "Integration Test User",
    "email": "integration@test.com",
    "password": "Integration123!"
  }' \
  -v

# Verify CORS headers are present in response
curl -H "Origin: http://localhost:8080" \
  -H "Access-Control-Request-Method: GET" \
  -H "Access-Control-Request-Headers: Content-Type" \
  -X OPTIONS http://localhost:8000/v1/users/get_all_pet_reports \
  -v 2>&1 | grep -i "access-control"
```

Step 4.2: Complete User Workflow Test

bash

1. Register new user

echo "Step 1: Registering user..."

```
curl -X POST http://localhost:8000/v1/users/signup \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Workflow Test User",
    "email": "workflow@test.com",
    "password": "Workflow123!"
  }' | jq '.'
```

2. Login and retrieve token

echo "Step 2: Logging in..."

```
TOKEN=$(curl -s -X POST http://localhost:8000/v1/users/signin \
  -H "Content-Type: application/json" \
  -d '{
    "email": "workflow@test.com",
    "password": "Workflow123!"
  }' | jq -r '.token')
```

echo "Token: \$TOKEN"

3. Create pet report

echo "Step 3: Creating pet report..."

```
curl -X POST http://localhost:8000/v1/users/create_pet_report \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "petName": "Buddy",
    "petType": "Dog",
    "breed": "Golden Retriever",
    "description": "Lost near Central Park",
    "lastSeenLocation": "Central Park, NY",
    "contactNumber": "1234567890"
  }' | jq '.'
```

4. Retrieve all reports

echo "Step 4: Retrieving all reports..."

```
curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/v1/users/get_all_pet_reports | jq '.'
```

Step 4.3: Network Connectivity Test

bash

```
# Test backend-database connectivity
docker exec petmanagement-backend ping -c 3 mongodb

# Test network configuration
docker network inspect petmanagement-backend-backend-network

# Verify DNS resolution
docker exec petmanagement-backend nslookup mongodb
```

Phase 5: Monitoring Setup

Step 5.1: Configure Monitoring System

```
bash

# Navigate to project root
cd ..

# Verify monitoring script exists
ls -la monitor-petfinder.sh

# Make script executable (if not already)
chmod +x monitor-petfinder.sh

# Review script configuration
head -30 monitor-petfinder.sh
```

Step 5.2: Initialize Monitoring

```
bash

# Run single health check
./monitor-petfinder.sh --check

# View monitoring output
cat /var/log/petfinder-alerts.log

# Start continuous monitoring in background
./monitor-petfinder.sh --daemon &

# Get monitoring process ID
ps aux | grep monitor-petfinder
```

Step 5.3: Verify Monitoring Functionality

```
bash
```

```
# Test error detection
for i in {1..6}; do
    echo "[$(date)] ERROR 500 Internal Server Error - Test error $i" >> /var/log/
done

# Trigger alert check
./monitor-petfinder.sh --check

# Verify alerts were generated
tail -10 /var/log/petfinder-alerts.log

# Test alert email (if configured)
./monitor-petfinder.sh --test-alert
```

Phase 6: Final Validation

Step 6.1: Complete System Health Check

```
bash

# Check all containers are running
docker ps --filter "name=petmanagement" --format "table {{.Names}}\t{{.Status}}"

# Verify all services are healthy
docker ps --filter "health=healthy" --filter "name=petmanagement"

# Check resource usage
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# Verify volumes are mounted correctly
docker volume ls | grep petmanagement
docker volume inspect mongodb_data
```

Step 6.2: End-to-End Application Test

```
bash
```

```
# 1. Frontend accessibility
curl -s -o /dev/null -w "Frontend Status: %{http_code}\n" http://localhost:8080

# 2. Backend API health
curl -s http://localhost:8000/ | jq -r '.message'

# 3. Database connectivity
docker exec petmanagement-backend-mongodb mongosh --eval "db.runCommand({connec

# 4. Complete user journey
echo "Testing complete user journey..."
./test-complete-workflow.sh # If you have a test script

# 5. Performance baseline
ab -n 100 -c 10 http://localhost:8000/ 2>/dev/null | grep -A 5 "Requests per se
```



Service Management

Backend Service Management

Basic Operations

```
bash

# Navigate to backend directory
cd petmanagement_backend_rajalakshmi

# Start services
docker-compose up -d

# Stop services
docker-compose down

# Restart services
docker-compose restart

# Stop specific service
docker-compose stop backend
docker-compose stop mongodb

# Start specific service
docker-compose start backend
docker-compose start mongodb
```

Advanced Operations

```
bash

# Rebuild and restart with no cache
docker-compose build --no-cache
docker-compose up -d --force-recreate

# View logs (real-time)
docker-compose logs -f

# View logs for specific service
docker-compose logs -f backend
docker-compose logs -f mongodb --tail=100

# Scale backend service (if configured)
docker-compose up -d --scale backend=3

# Execute command in running container
docker-compose exec backend npm run test
docker-compose exec mongodb mongosh
```

Frontend Service Management

Basic Operations

```
bash

# Navigate to frontend directory
cd petmanagement_frontend_rajalakshmi

# Start service
docker-compose up -d

# Stop service
docker-compose down

# Restart service
docker-compose restart

# View service status
docker-compose ps
```

Advanced Operations

```
bash
```

```
# Rebuild with latest changes
docker-compose build --no-cache frontend
docker-compose up -d --force-recreate

# View logs
docker-compose logs -f frontend

# Access Nginx container
docker-compose exec frontend sh

# Reload Nginx configuration
docker-compose exec frontend nginx -s reload

# Test Nginx configuration
docker-compose exec frontend nginx -t
```

Full Stack Management

Using Root docker-compose.yml

```
bash
```



```
# From project root directory
cd petmanagement_project

# Start entire stack
docker-compose up -d --build

# Stop entire stack
docker-compose down

# Stop and remove volumes (CAUTION: Data loss)
docker-compose down -v

# View all service logs
docker-compose logs -f

# View specific service logs
docker-compose logs -f frontend
docker-compose logs -f backend
docker-compose logs -f mongodb

# Check all service status
docker-compose ps

# Restart entire stack
docker-compose restart

# Restart specific service
docker-compose restart backend
```

Container Management Commands

```
bash
```

```
# List all containers
docker ps -a --filter "name=petmanagement"

# Stop all petmanagement containers
docker stop $(docker ps -q --filter "name=petmanagement")

# Remove all petmanagement containers
docker rm $(docker ps -aq --filter "name=petmanagement")

# View container resource usage
docker stats petmanagement-frontend petmanagement-backend petmanagement-backend

# Inspect container details
docker inspect petmanagement-backend | jq '.[0].State'

# View container logs
docker logs petmanagement-backend --tail=100 -f
docker logs petmanagement-frontend --tail=100 -f
docker logs petmanagement-backend-mongodb --tail=100 -f

# Execute commands in containers
docker exec -it petmanagement-backend sh
docker exec -it petmanagement-frontend sh
docker exec -it petmanagement-backend-mongodb mongosh
```

Volume Management

```
bash

# List volumes
docker volume ls | grep petmanagement

# Inspect volume
docker volume inspect mongodb_data

# Backup volume
docker run --rm -v mongodb_data:/data -v $(pwd):/backup alpine tar czf /backup/

# Restore volume
docker run --rm -v mongodb_data:/data -v $(pwd):/backup alpine tar xzf /backup/

# Remove unused volumes (CAUTION)
docker volume prune
```

Network Management

```
bash

# List networks
docker network ls | grep petmanagement

# Inspect network
docker network inspect backend-network

# Check connected containers
docker network inspect backend-network | jq '.[0].Containers'

# Create custom network
docker network create --driver bridge petmanagement-custom-network

# Connect container to network
docker network connect petmanagement-custom-network petmanagement-backend
```

Troubleshooting

Common Issues and Solutions

Issue 1: Backend Container Exits Immediately

Symptoms:

```
bash

docker-compose ps
# Shows backend container as "Exit 1" or "Exited"
```

Diagnosis:

```
bash

# Check backend logs
cd petmanagement_backend_rajalakshmi
docker-compose logs backend

# Common error messages:
# - "Cannot connect to MongoDB"
# - "Environment variable not found"
# - "Module not found"
```

Solutions:

```
bash
```

```
# Solution A: Database connection issue
```

```
# Verify MongoDB is running
```

```
docker-compose logs mongodb
```

```
docker exec petmanagement-backend-mongodb mongosh --eval "db.adminCommand('ping
```

```
# Solution B: Environment variables
```